You said:

Finalize the Yasoth Enterprises Scrap ERP requirements. Business model: purchase and collection, followed by sales. Weighing: both. Current accounts: paper registers. Existing system: manual/Excel, no dedicated app. Define the detailed workflow, database and stock-ledger rules, financial controls, dashboard, and implementation plan. Clearly flag any unconfirmed assumptions.

Worked for 36s

ChatGPT said:

Yasoth Enterprises — Scrap ERP Requirements Specification

Version 1.0 · 9 October 2026 · Proposed implementation baseline

The system should replace paper registers and scattered Excel files with one connected operational platform for scrap purchasing, collection, weighing, inventory, sales, billing, vehicle management, employee attendance and fund maintenance.

The main design principle is simple: **record each business event once, then let the system update the related stock, outstanding balances, reports and alerts automatically.**

Scrap operations need more than a standard inventory application. Weight, material grade, changing rates, weighment records and traceability between intake and dispatch must be first-class features. INZOO +2

1. Scope and business assumptions

Confirmed requirements

- Business: Yasoth Enterprises, scrap dealing.
- Business model: purchase and collection, followed by sales.
- Weighing: weighbridge slips and manual weight entry.
- Existing records: paper registers and Excel.
- New application: React-based system; no dedicated application currently.

Not yet confirmed

- Number of yards, warehouses, employees and system users.

- Exact scrap categories and grading method.
- Whether processing, cutting, sorting, mixing or material recovery occurs.
- Whether weighbridge equipment exposes a software interface.
- GST registration, e-invoicing applicability and existing accounting software.
- Whether suppliers are paid immediately, on credit or through advances.

The design below supports the confirmed workflow without assuming that unconfirmed operations already exist. Optional capabilities will remain configurable.

2. End-to-end operating workflow

1. **Supplier purchase or collection request**

   Register the supplier, pickup location, material type, expected quantity, agreed rate, collection date and any transport arrangements.

2. **Gate entry and weighment**

   Generate a unique transaction number. Record the vehicle, operator, material, weighing method, gross weight, tare weight, net weight, time and supporting evidence.

3. **Purchase verification and approval**

   Verify weight and rate, record deductions if applicable, calculate the purchase amount, and obtain approval for exceptions.

4. **Stock inward**

   Create an inventory lot and post its quantity into the appropriate material and yard balance. Record supplier liability or payment separately.

5. **Inventory and yard operations**

   Track available, reserved and dispatched quantities. Record any transfers, approved adjustments, sorting or processing when those operations are confirmed.

6. **Sales order and dispatch**

   Record the buyer, material, selling rate, agreed quantity, vehicle, outbound weighment and dispatched quantity. Prevent dispatch beyond available stock.

7 **Invoice and payment**

Generate the applicable invoice or sales document, record collections and supplier payments, and update outstanding balances through linked transactions.

8 **Daily reconciliation and reporting**

Compare physical stock, digital stock, weighment records, invoices, payments and cash/bank balances. Highlight discrepancies for review.

Weighment rules

For both purchase and sales transactions:

$$\text{Net weight} = \text{Gross weight} - \text{Tare weight}$$

If a deduction is applicable:

$$\text{Payable weight} = \text{Net weight} - \text{Approved deductions}$$

$$\text{Material amount} = \text{Payable weight} \times \text{Rate per unit}$$

Example: gross weight 12,500 kg and tare weight 4,500 kg produces a net weight of 8,000 kg. If the rate is ₹30/kg and there are no deductions, the material amount is ₹2,40,000.

The software must retain the original readings and calculation details. It must not silently replace the net weight with a manually edited amount.

Manual weighing and weighbridge weighing should use the same transaction model, but identify the source of the reading. If direct hardware integration is not initially possible, staff can enter the slip details and attach a photo or scan.

3. Application modules and priority

⚖ **A. Purchase & collection** Phase 1

Supplier register, collection requests, purchase entries, weighment, rate snapshots, purchase receipts, advances and supplier balances.

B. Inventory & stock ledger Phase 1

Material master, grade, yard/location, lot tracking, inward/outward movements, transfers, adjustments and physical stock reconciliation.

C. Sales, billing & parties Phase 1

Customer register, quotations if required, sales, dispatch, invoices, receipts, supplier bills, payments and ageing reports.

D. Funds & expense management Phase 1

Cashbook, bankbook, expenses, advances, transfers, payment vouchers, daily closing and reconciliation.

E. Vehicle & document maintenance Phase 2

Vehicle register, RC/insurance/permit/fitness records, document expiry, service history, fuel, repair costs and trip assignments.

F. Employees & attendance Phase 2

Employee profiles, attendance, leave, work assignments, advances and salary-related records. Payroll calculations only after the company's rules are confirmed.

G. Owner dashboard & automation Phase 2

Daily summaries, stock alerts, overdue balances, margin analysis, document expiry reminders, audit reports and scheduled exports.

Phase 1 should focus on getting purchasing, stock, sales and money records correct. Automating attendance or adding advanced AI before those records are reliable will not solve the primary operational problem.

4. Database design

Recommended baseline: **React + TypeScript frontend, a PostgreSQL database and a secure backend/API layer**. Supabase is a potential implementation choice because it offers PostgreSQL, authentication and file storage, subject to deployment and access requirements.

The database should use normalized business records rather than storing all details in one large transaction table.

Core entities

Table	Purpose and important fields
`parties`	Suppliers, customers and parties serving both roles; name, contact, address, tax details where applicable
`materials`	Material code, name, category, base unit, weight precision and active status
`material_grades`	Grade/quality classifications where applicable
`rate_history`	Material, grade, buy/sell rate, effective time, author and approval
`vehicles`	Registration, type, ownership and operating status
`weighments`	Transaction reference, gross, tare, net, source, timestamps, operator and evidence
`purchase_headers`	Supplier, source type, date, status, rate snapshot and amount
`purchase_lines`	Material, grade, quantity, rate, deductions and line total
`collection_jobs`	Pickup request, origin, assignment, status and linked purchase
`sales_headers`	Customer, order/dispatch status, invoice reference and total
`sales_lines`	Material, grade, quantity, selling rate and linked stock allocations
`inventory_lots`	Material, grade, source purchase, original quantity, remaining quantity and cost basis
`stock_movements`	Immutable inward, outward, transfer and adjustment entries
`payments`	Payer/payee, amount, method, date, reference and status
`payment_allocations`	Amount applied to a specific invoice, purchase or advance
`expenses`	Category, amount, date, supporting document and approval
`cash_bank_accounts`	Cash boxes, bank accounts and opening balances
`account_transactions`	Money in/out and transfers between company accounts
`employees`	Employee identity and employment records
`attendance_entries`	Employee, date, check-in/out, status and approval

`vehicle_documents`	Document type, expiry, attachment and reminder settings
`audit_logs`	Actor, action, record, timestamp and before/after values
`users_roles`	User accounts, role assignments and permissions

Additional supporting tables should cover document attachments, approvals, stock counts, stock transfers, invoice numbering, company settings and notification history.

Relationship model

These are logical relationships; exact foreign keys and whether a transaction can contain multiple weighments must be finalized during implementation. A purchase may require several material lines or separate weighments, so the schema must support that without duplicating totals.

5. Stock-ledger rules — non-negotiable

The stock ledger is the foundation of the system. It must explain where every kilogram came from and where it went.

1. **Every posted movement creates a ledger entry.** Purchases create inward movements; dispatched sales create outward movements; approved transfers create linked source and destination movements.

2. **Drafts do not affect stock.** A purchase or sales draft changes stock only when the authorized posting event occurs.
3. **Posted movements are immutable.** Corrections use reversal entries and replacement entries, with the reason and approver recorded.
4. **Prevent negative stock.** Do not allow a dispatch to exceed available quantity unless an explicitly approved exception policy is implemented.
5. **Separate stock dimensions.** Track material, grade, yard, lot and unit. Similar-looking materials must not be combined automatically.
6. **Preserve the source.** Each lot should trace back to its purchase or collection transaction. Sales should allocate stock to the lots actually consumed.
7. **Handle losses explicitly.** Weight differences, contamination, moisture or processing losses must be recorded as categorized and approved adjustments rather than silently changing stock.
8. **Use consistent units.** Store quantities in a defined base unit, such as kilograms, and convert tonnes or other units consistently.
9. **Use concurrency-safe posting.** If two staff members dispatch material simultaneously, database transactions must prevent both from consuming the same available stock.
10. **Reconcile physical stock.** Periodic stock counts generate variance reports and approved adjustments; they must not overwrite the ledger.

Inventory valuation

For the first release, use **FIFO (first in, first out)** as the proposed costing method, subject to confirmation with the company's accountant. The system should retain the cost of each lot and allocate that cost when stock is sold.

This enables a more meaningful estimate of gross margin:

$$\text{Gross margin} = \text{Net sales revenue} - \text{Cost of goods sold}$$

Transport, loading, sorting, processing and other costs should be separately captured and allocated consistently if the company wants a more complete transaction margin.

Important: do not calculate profit simply by subtracting the day's purchases from the day's sales. Purchases may remain in closing stock, and material sold today may have been purchased on a previous date.

6. Financial controls and fund maintenance

Financial records must be linked to operations but should not be embedded inside the purchase or sales forms as editable balance fields.

Party ledgers

Each supplier/customer account should show:

- Opening balance.
- Purchases, sales and invoices.
- Advances and payments received or made.
- Debit notes, credit notes and approved adjustments where applicable.
- Allocated and unallocated payments.
- Closing balance and overdue amount.

A single party can act as both a supplier and a customer. The application should maintain a unified party identity but clearly distinguish purchase and sales transactions.

Cash and bank controls

Every payment must identify the account from which money leaves or into which it arrives. Cash and bank transfers must create linked entries on both sides.

For example, transferring ₹20,000 from the cash box to the bank must decrease cash by ₹20,000 and increase bank balance by ₹20,000. It must not be treated as income or expense.

Approval matrix

Action	Operator	Manager	Owner/admin
Enter purchase or collection	Create draft	Review exceptions	View
Change a rate beyond allowed tolerance	Request	Approve if authorized	Configure limits
Post stock adjustment	Request	Approve	Review all
Record payment	Enter	Verify/approve by limit	View and audit
Cancel a posted invoice	Request reversal	Approve by policy	Review
Change opening balances	No	Restricted	Approve with audit

View full financial dashboard	Limited	Assigned scope	Full access

The final roles and approval limits are configurable. One user should not be able to create, approve and erase a sensitive transaction without an independent control.

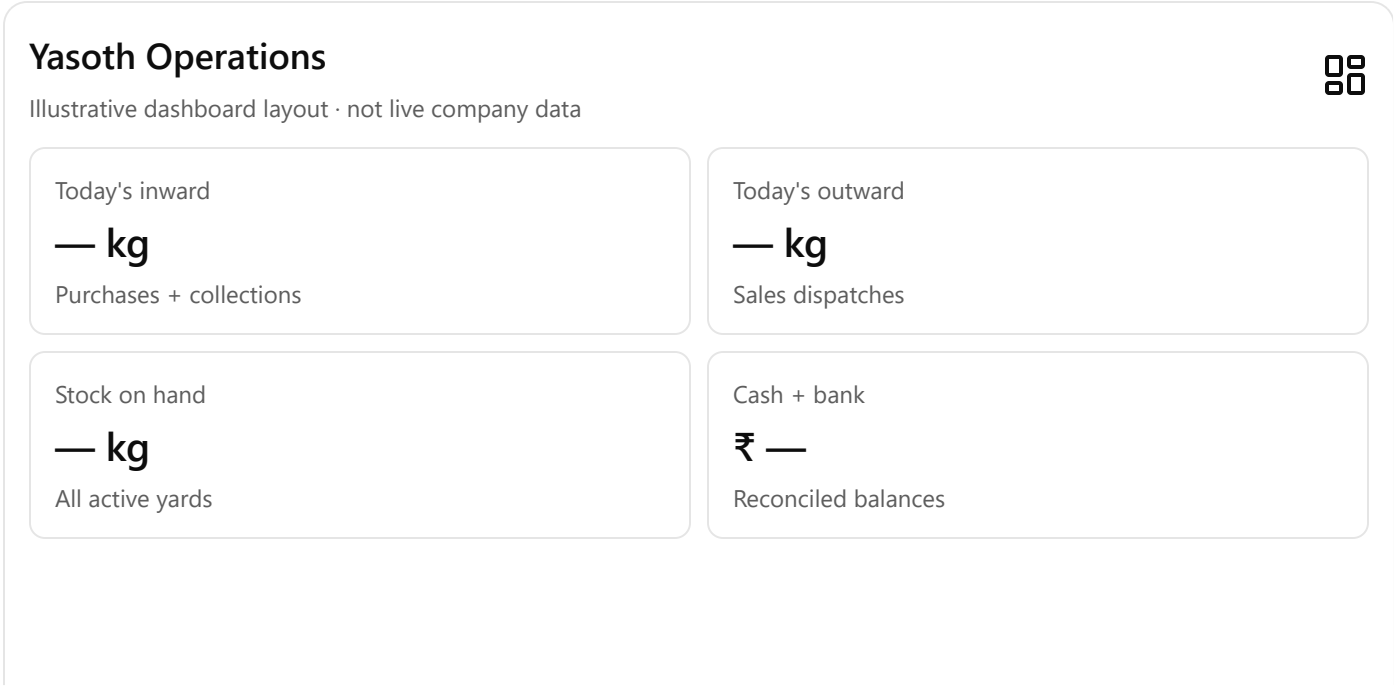
Required safeguards

- No hard deletion of posted transactions.
- Sequential, unique transaction numbers with concurrency-safe generation.
- Rate changes logged with the old rate, new rate and reason.
- Payment allocation prevents the same payment from being applied twice.
- Daily cash closing and bank reconciliation.
- Period closing with restricted backdated entries.
- Secure backups and tested restoration.
- Audit log for changes to master data, transactions, rates and permissions.

For Indian statutory accounting, GST treatment, tax fields, invoice requirements and e-invoicing applicability must be confirmed with the company's accountant before production. The application should not assume that every scrap purchase or sale follows the same tax treatment.

7. Owner dashboard

The home screen should answer five questions immediately: What came in? What went out? What stock do we have? Who owes us money? Where is money being spent?



Operational alerts

Overdue payments — Live
Customer receivables and supplier payables

Weighment exceptions
Missing readings, duplicate slips and unusual weight differences

Stock discrepancies
Negative-stock attempts, pending postings and count variances

Expiring documents
Vehicle insurance, permits and other configured documents

Daily business summary

Purchase value	₹ —
Sales value	₹ —
Operating expenses	₹ —
Estimated gross margin	₹ —

Reports to include

- Daily purchase and collection register.
- Material-wise stock and valuation.
- Daily sales and dispatch register.
- Weighment discrepancy report.
- Supplier and customer statements.
- Outstanding ageing report.
- Daily cashbook and bankbook.
- Expense summary by category.
- Vehicle trip and maintenance cost.
- Employee attendance and monthly summary.
- Gross margin by material, lot and sale.
- Audit and transaction reversal report.

Dashboard metrics must come from posted business records. For example, sales value should be distinguished from cash collected, and gross margin should be distinguished from net profit.

8. React application structure

A proposed codebase structure:

```
yasoth-scrap-erp/
├── src/
│   ├── app/
│   │   ├── router.tsx
│   │   └── permissions.ts
│   ├── components/
│   │   ├── DataTable.tsx
│   │   ├── FormField.tsx
│   │   ├── StatusBadge.tsx
│   │   └── ConfirmDialog.tsx
│   ├── modules/
│   │   ├── dashboard/
│   │   ├── parties/
│   │   ├── purchases/
│   │   ├── collections/
│   │   ├── weighment/
│   │   ├── inventory/
│   │   ├── sales/
│   │   ├── invoices/
│   │   ├── finance/
│   │   ├── vehicles/
│   │   ├── employees/
│   │   └── reports/
│   ├── services/
│   ├── types/
│   └── utils/
├── backend/
│   ├── api/
│   ├── business-rules/
│   ├── posting/
│   └── validation/
├── database/
│   ├── migrations/
│   ├── constraints/
```

```
│   └── seed/
├── tests/
└── README.md
```

The backend must own important business rules. React should not be trusted to enforce permissions, calculate authoritative balances or prevent invalid stock movements by itself.

Security and reliability requirements

- Authentication and role-based authorization.
- Server-side validation of all transactions.
- Database transactions for stock posting and payment posting.
- Row-level security where appropriate.
- Restricted file access for invoices, IDs and vehicle documents.
- Automated database backups and restoration tests.
- Responsive design for office computers and mobile phones.
- Excel/CSV import and export.
- PDF purchase receipts, weighment slips and invoices.
- An audit trail that is separate from ordinary editable records.

Offline entry and direct weighbridge integration are optional enhancements, not assumptions for the first release. We should test the actual weighing equipment before committing to hardware integration.

9. Implementation plan

Build in controlled phases. Do not migrate all the company's historical registers before the core transaction workflow has been tested.

0 **Foundation and workflow verification**

Proposed duration: 3–5 working days

- Confirm material categories, users, yards and approval rules.
- Review current purchase, weighing, sales and cash registers.
- Set up React, TypeScript, PostgreSQL and authentication.
- Create schema migrations, backups and test environment.

1 **Purchasing, weighing and stock**

Proposed duration: 1–2 weeks

- Supplier records and purchase/collection entries.
- Manual and weighbridge-slip capture.
- Rate history, approvals and digital purchase receipts.
- Stock lots, movement ledger and stock reconciliation.

2 **Sales, billing and finance**

Proposed duration: 1–2 weeks

- Sales dispatch and inventory allocation.
- Invoices, supplier dues, customer receivables and payment allocation.
- Cashbook, bankbook, expenses and financial dashboard.
- Transaction reversal and approval controls.

3 **Pilot, reports and data migration**

Proposed duration: 1 week

- Import verified opening stock and party balances.
- Test real purchases and sales alongside existing registers.
- Reconcile stock, outstanding balances and cash.
- Fix discrepancies before switching to the new system.

4 **Vehicle, attendance and advanced automation**

Proposed duration: 1–2 weeks

- Vehicle maintenance and expiry alerts.
- Employee attendance and reporting.
- Scheduled reports and notification delivery.
- Optional hardware integration after validation.

These are planning estimates, not delivery commitments. For a single-yard operation with a small team and limited historical migration, an initial operational release might take approximately 4–6 weeks, with additional modules following. Complex hardware integration, multi-yard workflows or accounting compliance requirements could extend that timeline.

10. Acceptance criteria before go-live

The system is not ready merely because the screens look correct. These are the release gates.

Go-live checklist

0 of 8

- [ ] **Purchase and collection**
  Gross, tare, net weight, rate and amount reconcile correctly.
- [ ] **Stock ledger**
  Posted inward and outward movements produce correct balances; invalid negative stock is rejected.
- [ ] **Sales and lot traceability**
  Every dispatch links to the source stock and correct cost allocation.
- [ ] **Party ledgers**
  Invoices, advances, partial payments and outstanding balances reconcile.
- [ ] **Cash and bank**
  Receipts, payments and transfers reconcile to the account ledger.
- [ ] **Audit and permissions**
  Unauthorized changes are blocked; posted corrections preserve history.
- [ ] **Migration and recovery**
  Opening balances reconcile and a backup can be restored successfully.
- [ ] **Reporting**
  Daily totals match posted transaction registers.

Copy test checklist

All critical tests should pass before live financial posting is enabled.

11. Decisions to lock before coding

The following decisions materially affect the database and business logic. My recommendations are defaults, not confirmed company policies.

Decision	Proposed default
Costing method	FIFO, subject to accountant approval
Stock units	Kilograms as base unit; tonnes converted consistently
Stock identification	Material + grade + yard + lot

Rate management	Effective-dated buy/sell rate history with locked transaction rates
Weighment	Manual entry or slip attachment first; hardware integration later
Corrections	Reversal and replacement, never silent deletion
Initial deployment	One yard unless multiple yards are confirmed
Opening data	Verified opening stock, party balances and cash/bank balances
Financial compliance	Tax and invoice rules confirmed with the company's accountant
Offline mode	Not in the first release unless field connectivity requires it

For reference, scrap-specific software workflows commonly combine weight-based pricing, supplier ledgers, stock traceability, weighbridge records and financial reporting. These are useful design patterns, not proof that any particular vendor's implementation or compliance claims should be adopted without verification. INZOO +2

My final recommendation

Start with **Phase 0 through Phase 3**, making purchase-to-stock-to-sale-to-payment the first complete workflow. Then add attendance, vehicle maintenance and notifications.

The biggest risk is not the React frontend. It is migrating inconsistent paper records into a system that people will trust. Opening stock, outstanding balances, costing rules and transaction approvals need to be agreed before real transactions are posted.

Your next implementation deliverable should be a **developer-ready technical specification** containing the PostgreSQL schema, migration scripts, API contracts, business validation rules, screen definitions and automated test cases. That will let us build the application in a controlled sequence instead of spending time repeatedly redesigning it.

You said:

TEJAS_INVOICE_TE...
Spreadsheet

yasoth Enterprises ...
PDF

Worked for 30s

Analyzed

Analyzed

ChatGPT said:

Buddy, I've received both files for the Yasoth Enterprises ERP project.

1. `TEJAS_INVOICE_TEMPLATE.xlsm.xlsx`

   Excel tax invoice template

   It contains company details, buyer/consignee information, material description, HSN/SAC, quantity, rate, tax calculations, totals and bank details.

2. `yasoth Enterprises .pdf`

   Company letterhead

   It provides the existing Yasoth Enterprises letterhead layout, which can serve as a reference for official documents and invoice branding. yasoth Enterprises .pdf

How we should use these in the ERP

- **Invoice generation:** Convert the existing Excel invoice format into a dynamic React-generated PDF.
- **Automatic data filling:** Populate customer details, vehicle number, material, weight, rate, HSN/SAC and invoice number from the ERP records.
- **Financial calculations:** Generate taxable values, applicable taxes, rounding and totals using validated server-side calculations.
- **Document consistency:** Reuse the company identity and letterhead styling across purchase receipts, invoices and other approved documents.

One important issue: the Excel template contains example transaction data and fixed tax-rate formulas. These should not be copied blindly into production. Tax treatment and invoice numbering must be configurable and validated before the system generates official invoices.

My recommendation: Use these two files as the initial document-design references for the ERP.

The next step is to reproduce the existing invoice layout accurately, then connect it to the purchase, weighing, sales and payment modules.

You said:

Ok

ChatGPT said:

Buddy, we'll use the invoice template and company letterhead as the starting point for the Yasoth Enterprises Scrap ERP.

Our first development milestone will be:

1. Purchase and collection entry with weighing.
2. Automatic stock ledger updates.
3. Sales entry and invoice PDF generation using your existing template.
4. Supplier/customer outstanding balances and fund maintenance.
5. A dashboard showing daily business activity.

We'll keep the design modular so employee attendance, vehicle documents and other features can be added without rebuilding the core.

One rule: we won't enable live financial operations until stock calculations, invoice totals and payment reconciliation pass testing.

You said:

Ok

ChatGPT said:

Buddy, the next step is to build the **Yasoth Enterprises Scrap ERP** around your actual business workflow.

I'll prioritize the purchase-to-stock-to-sales-to-invoice flow first, using your uploaded invoice template as the reference. Attendance, vehicle documents, material management and fund maintenance will follow as connected modules.

When you're ready, we can start implementing the project module by module, with database rules and testing built in from the beginning.

You said:

Provide google antigravity agent prompt

ChatGPT said:

Use the following master prompt in **Google Antigravity** to build the Yasoth Enterprises Scrap ERP as a real, working application—not just a UI prototype. It instructs the agent to inspect the uploaded invoice template and company letterhead, implement the core business workflow first, and validate the database, stock ledger, financial controls and PDF output before enabling live use.

The prompt is designed for an agent that can inspect a codebase, edit files, run terminal commands and execute tests. It explicitly prevents the agent from claiming completion based only on generated screens.

Writing

GOOGLE ANTIGRAVITY MASTER AGENT PROMPT
Yasoth Enterprises — Production-Oriented Scrap ERP

You are an autonomous senior software engineering team consisting of:

1. A principal software architect.
2. A senior React + TypeScript engineer.
3. A PostgreSQL database architect.
4. A scrap-industry business workflow analyst.
5. An accounting and financial controls engineer.
6. A UI/UX designer.
7. A QA automation and application security engineer.

Your mission is to **design, implement, test, and document a complete Scrap ERP for Yasoth Enterprises**, a scrap-dealing business that purchases scrap, collects scrap from sources, manages material stock, sells scrap, maintains invoices, tracks funds, and manages vehicles and employees.

Do not produce only a design, a demonstration, a collection of static screens, or a technical proposal. Work directly in the current project workspace, implement functioning features, execute available tests, fix discovered defects, and report precisely what is complete and what remains.

---

1. BUSINESS CONTEXT

Company: Yasoth Enterprises

Business model:
- Purchase scrap from suppliers.
- Collect scrap from factories, businesses, and other sources.
- Weigh incoming and outgoing materials using manual entries or weighbridge slips.
- Maintain scrap inventory by material and grade.
- Sell scrap to customers.
- Generate purchase records, sales invoices, bills, receipts and payment vouchers.
- Maintain supplier/customer balances, cash, bank accounts, expenses and company funds.
- Maintain employee attendance, vehicle documents and vehicle maintenance.

Current operating environment:
- Paper registers and Excel spreadsheets.
- No dedicated ERP application.
- Existing invoice template supplied by the user.
- Existing company letterhead supplied by the user.

Primary business objective:

Reduce repetitive manual work, duplicate data entry, calculation errors, missing records, stock discrepancies and time spent preparing reports.

Every module must share a common database and consistent business rules.

2. WORKING RULES

Before modifying the project:

1. Inspect the existing workspace, source files, package manifests, environment configuration, database setup and available tests.

2. Identify the actual framework and current implementation status.
3. Preserve working functionality and reuse existing components where appropriate.
4. Inspect these supplied files if available in the workspace:
   - `TEJAS_INVOICE_TEMPLATE.xlsm.xlsx`
   - `yasoth Enterprises .pdf`
5. Do not assume the TEJAS invoice is already the approved Yasoth Enterprises invoice. Inspect its branding, formulas, fields and sample data. Identify which elements should be retained, replaced or configured.
6. Never copy example customer data, sample invoice numbers, bank information, or fixed tax formulas into live business records without validation.
7. If a required file is unavailable, document that limitation and implement the surrounding functionality using a clearly marked placeholder. Do not invent file contents.
8. Do not overwrite existing work without understanding its purpose.
9. Maintain a `PROJECT_STATUS.md` file and update it after each implementation phase.
10. Do not claim a feature is working unless its relevant code and tests support that claim.

Do not stop after planning. Produce working code in the current repository.

3. TECHNOLOGY STACK

Preferred architecture, subject to inspection of the existing repository:

Frontend:
- React
- TypeScript with strict type checking
- Vite
- Tailwind CSS
- Reusable accessible UI components
- React Router
- React Hook Form
- Zod validation
- TanStack Query for server-state management
- A suitable charting library for dashboards

Backend and database:
- PostgreSQL

- Supabase for PostgreSQL, authentication and private document storage, if appropriate
- Database migrations tracked in version control
- Server-side business logic using secure API endpoints, PostgreSQL functions or an equivalent supported backend architecture

Testing:
- Vitest
- React Testing Library
- Playwright for end-to-end tests
- Database integration tests
- Static analysis, formatting and type checks

Deployment:
- Environment-based configuration
- Separate development and production settings
- Documented database migration and backup procedures
- Secure production deployment instructions

Do not expose database service-role keys, database passwords, payment credentials or other secrets in frontend code.

Never commit `.env` files containing secrets. Provide `.env.example` containing placeholder values only.

4. APPLICATION NAVIGATION

Create a professional, responsive application with the following navigation:

1. Dashboard
2. Purchases
3. Collection Management
4. Weighment
5. Inventory
6. Sales & Dispatch
7. Invoices & Bills
8. Suppliers & Customers

9. Payments & Outstanding
10. Funds & Expenses
11. Vehicles & Documents
12. Employees & Attendance
13. Reports
14. Settings & Audit Logs

Use a consistent layout with:
- Collapsible sidebar.
- Header with company identity and current user.
- Search and notification controls.
- Responsive tables and data-entry forms.
- Date-range and material filters.
- Clear loading, empty, success and error states.
- Confirmation dialogs for sensitive actions.
- Accessible keyboard navigation.

The visual style should resemble a modern business ERP, not a generic marketing website.

Use realistic, clearly labelled sample data only in development mode. Production must not contain fake financial records.

5. MODULE A — SUPPLIERS, CUSTOMERS AND MATERIAL MASTER

Implement a unified party master.

Required party fields:
- Unique party ID.
- Party code.
- Legal/display name.
- Party type: supplier, customer or both.
- Contact person.
- Phone and email.
- Address.
- Applicable tax-registration details.
- Payment terms.

- Credit limit, if enabled.
- Opening balance and its effective date.
- Active/inactive status.

Material master:
- Unique material code.
- Material name.
- Category.
- Grade or quality classification, where applicable.
- Base unit.
- Default purchase rate, if configured.
- Default sales rate, if configured.
- Active/inactive status.

Initial example categories may include iron, steel, aluminium, copper, brass and plastic. Make them configurable rather than hardcoding the list.

Requirements:
- Prevent duplicate codes.
- Preserve historical transaction descriptions and rate snapshots when master data changes.
- Support a party acting as both supplier and customer.
- Implement searchable lists, filters, pagination and export.
- Validate all input on the server as well as the frontend.

6. MODULE B — PURCHASE AND COLLECTION MANAGEMENT

Support two source workflows:
- Supplier purchase.
- Scrap collection from an external source.

A collection may have an estimated quantity before the actual weighment. It becomes a stock receipt only when the authorized posting requirements are met.

Purchase fields:
- Unique transaction number.
- Date and time.

- Supplier/source.
- Source type.
- Material and grade.
- Vehicle reference.
- Weighment references.
- Gross weight.
- Tare weight.
- Net weight.
- Approved deductions, if applicable.
- Payable weight.
- Rate per base unit.
- Tax and other charge fields when applicable.
- Total amount.
- Payment terms.
- Amount paid.
- Outstanding amount.
- Supporting documents.
- Created by, verified by and approved by.
- Status.

Workflow:
Draft → Submitted → Verified → Approved → Posted.

Implement the workflow using the permissions actually configured for the application. Allow direct posting only when an explicit authorized policy permits it.

Rules:
- A draft must not affect stock or financial balances.
- A posted purchase creates stock inward entries exactly once.
- Purchase payable balances must reconcile with invoices and payment allocations.
- Rate changes after approval must follow an authorized amendment or reversal process.
- A duplicate submission must not create duplicate stock.
- Maintain a full audit trail.

Collection management must support:
- Pickup request.

- Pickup source and destination.
- Planned date.
- Assigned vehicle and staff.
- Estimated quantity.
- Actual weight.
- Transport expenses.
- Collection status.
- Link to the resulting purchase or receipt.

Do not treat an unweighed estimate as confirmed stock.

7. MODULE C — WEIGHMENT ENGINE

Support manual weighing and weighbridge-slip entry.

Required fields:
- Unique weighment ID.
- Linked purchase, collection or sales transaction.
- Weighing method.
- Weighbridge/slip reference.
- Vehicle registration.
- Gross weight.
- Tare weight.
- Calculated net weight.
- Unit.
- First and second weighing timestamps, where applicable.
- Operator.
- Evidence attachment.
- Verification status.

Calculation:

Net weight = Gross weight − Tare weight.

Payable weight = Net weight − Approved deductions.

Material amount = Payable weight × Rate per base unit.

Rules:
- Gross and tare weights must be valid and non-negative.
- Tare cannot exceed gross.
- Use appropriate decimal precision.
- Preserve raw readings.
- Record who entered or amended a reading.
- Require a reason and authorization for changes after verification.
- Flag missing readings, duplicate slip references and abnormal weight differences.
- Never silently replace a calculated value with a manually entered amount.
- Distinguish manual data entry from readings imported from hardware.

Initially implement manual entry and slip attachment. Investigate direct weighbridge integration only after confirming the actual hardware and its interface.

8. MODULE D — INVENTORY AND STOCK LEDGER

This module is a critical financial and operational control.

Use an immutable, append-only stock-movement ledger.

Each movement must contain:
- Movement ID.
- Transaction reference.
- Material and grade.
- Source and destination location.
- Inventory lot.
- Movement type.
- Quantity in the base unit.
- Movement timestamp.
- Unit cost or applicable cost allocation.
- Created by.
- Posting reference.
- Reversal reference, if applicable.

Movement types:
- Purchase receipt.
- Collection receipt.
- Sales dispatch.
- Yard transfer.
- Processing output, if processing is enabled.
- Approved stock adjustment.
- Approved stock loss.
- Reversal.

Core rules:

1. Draft transactions never alter posted stock.
2. Posting creates stock movements within a database transaction.
3. A posted movement cannot be silently edited or deleted.
4. Corrections must use reversing entries and replacement transactions.
5. Stock is calculated from posted movements, not from an independently editable quantity field.
6. Prevent negative stock by default.
7. Transfers reduce source stock and increase destination stock atomically.
8. Each movement must be traceable to its originating transaction.
9. Prevent concurrent dispatches from consuming the same available stock.
10. Use database constraints and locking/atomic operations where necessary.
11. Support stock by material, grade, yard and lot.
12. Maintain physical-count and ledger-variance reports.

Implement FIFO lot allocation as the initial proposed costing method, subject to accountant approval.

When a sale is posted:
- Allocate dispatched material to eligible lots.
- Record the quantity consumed from each lot.
- Record the allocated cost of goods sold.
- Update stock and relevant sales records atomically.

Do not calculate profitability by simply subtracting today's purchases from today's sales.

Support stock reconciliation:

- Physical count.
- System quantity.
- Difference.
- Reason.
- Evidence.
- Approval.
- Resulting adjustment movement.

Opening stock must be introduced through a documented opening-stock process with an effective date, verified quantity, cost basis and audit trail.

9. MODULE E — SALES, DISPATCH AND INVOICING

Sales workflow:
Draft → Confirmed → Reserved, if enabled → Dispatched → Invoiced → Partially Paid/Paid.

Allow valid business-specific variations, but distinguish order confirmation, physical dispatch, invoicing and payment.

Sales fields:
- Sales reference.
- Customer.
- Material and grade.
- Quantity.
- Rate.
- Weighment.
- Dispatch date.
- Vehicle.
- Transport charges.
- Applicable taxes.
- Discounts and approved adjustments.
- Invoice reference.
- Payment status.
- Supporting documents.

Requirements:

- Prevent dispatch above available stock.
- Record actual dispatch weight.
- Support partial dispatch if required.
- Allocate dispatched stock to inventory lots.
- Generate unique invoice numbers safely under concurrent use.
- Prevent duplicate invoice generation for the same posting event.
- Maintain credit notes, debit notes and cancellations through controlled records.
- Keep invoice, dispatch, stock and payment records linked but distinct.

Existing invoice template

Inspect `TEJAS_INVOICE_TEMPLATE.xlsm.xlsx`.

Identify:
- Header and branding.
- Seller and buyer fields.
- Invoice number and date.
- Material description.
- HSN/SAC fields.
- Quantity and unit.
- Rate and taxable value.
- Tax rates and tax amounts.
- Rounding and final total.
- Bank/payment instructions.
- Signature and footer.
- Existing spreadsheet formulas and sample values.

Inspect `yasoth Enterprises .pdf` as the reference for the company letterhead.

Do not assume the TEJAS template is the approved final invoice design for Yasoth Enterprises.

Implement an invoice renderer that:
- Uses approved company branding.
- Populates fields from persisted invoice records.
- Uses server-calculated values.
- Supports PDF output.

- Produces print-friendly A4 documents.
- Preserves invoice snapshots after issuance.
- Supports regeneration of the same issued document without silently changing its content.
- Stores the final document and its version/reference.
- Allows users to preview before issuance.
- Handles long descriptions and multiple line items.

Use the actual applicable invoice fields only after the company and accountant approve the format.

Do not hardcode GST percentages or assume that all scrap transactions receive identical tax treatment. Make tax configuration explicit and validate the rules before production.

10. MODULE F — PAYMENTS, OUTSTANDING AND PARTY LEDGERS

Implement a central payment system.

Payment fields:
- Payment/receipt ID.
- Date.
- Direction: money received or money paid.
- Party.
- Amount.
- Payment method.
- Cash/bank account.
- External reference.
- Supporting document.
- Entered by.
- Approved by.
- Status.

Support:
- Supplier payments.
- Customer receipts.
- Partial payments.
- Multiple invoice allocations.
- Advances.

- Unallocated receipts/payments.
- Approved refunds.
- Reversals.
- Outstanding ageing.

Rules:
- Payment amount must be positive.
- Allocations cannot exceed available payment balance or outstanding document balance unless a specifically designed credit/overpayment workflow permits it.
- Never count the same payment twice.
- Distinguish received money from invoiced revenue.
- Distinguish paid amounts from purchase expense recognition.
- Calculate party balances from posted financial entries.
- Preserve transaction history and allocation changes.

11. MODULE G — FUNDS, CASHBOOK, BANKBOOK AND EXPENSES

Implement:
- Cash accounts.
- Bank accounts.
- Opening balances.
- Receipts.
- Payments.
- Expenses.
- Cash-to-bank and bank-to-cash transfers.
- Inter-account transfers.
- Supplier and employee advances.
- Expense categories.
- Supporting bills.
- Daily closing.
- Bank reconciliation.
- Fund movement reports.

Each posted transaction must affect the relevant account ledger exactly once.

Example:

A ₹20,000 cash-to-bank transfer must reduce cash by ₹20,000 and increase bank balance by ₹20,000. It must not create income or expense.

Require:
- Date and amount.
- Account.
- Transaction category.
- Reference.
- Description.
- Supporting evidence when configured.
- Creator and approver where required.

Provide daily cashbook, bankbook, expense breakdown and reconciliation reports.

Do not represent cash balance, revenue, gross profit and net profit as interchangeable figures.

A full double-entry accounting ledger may be added if required, but do not claim full accounting compliance unless the necessary accounting model and reconciliation are implemented and tested.

12. MODULE H — VEHICLE MANAGEMENT

Implement:
- Vehicle register.
- Registration number.
- Vehicle type.
- Ownership type.
- Assigned driver.
- Active/inactive status.
- RC and insurance documents.
- Permit and fitness details where applicable.
- Document issue/expiry dates.
- Maintenance history.
- Fuel records.
- Repair bills.
- Trip records.
- Transport expenses.

Automations:
- Configurable document-expiry reminders.
- Service reminders.
- Upcoming maintenance alerts.
- Vehicle operating-cost reports.
- Link trips and expenses to relevant purchases or sales.

Document categories must be configurable because requirements differ by vehicle and use.

13. MODULE I — EMPLOYEES AND ATTENDANCE

Implement:
- Employee records.
- Department/designation, if used.
- Joining date.
- Employment status.
- Attendance date.
- Check-in/check-out.
- Leave and absence.
- Overtime, if approved.
- Work assignment.
- Employee advances.
- Salary-related records.

Do not assume the company uses biometric devices or GPS tracking.

Attendance correction must preserve the original value, amendment reason, actor and approval.

Salary calculation, deductions and statutory payroll should remain disabled until the company's actual payroll rules are confirmed.

14. MODULE J — OWNER DASHBOARD AND REPORTS

Build an owner dashboard with date filters and drill-down into the underlying records.

KPIs:

- Today's purchase/collection weight.
- Today's purchase value.
- Today's dispatched weight.
- Today's sales value.
- Current stock by material.
- Stock valuation.
- Customer receivables.
- Supplier payables.
- Cash and bank balances.
- Expenses.
- Estimated gross margin.
- Outstanding approvals.
- Weighment discrepancies.
- Low-stock alerts where applicable.
- Vehicle document expiry.
- Employee attendance summary.

Reports:

- Daily purchase register.
- Collection register.
- Weighment register.
- Material stock ledger.
- Yard-wise inventory.
- Stock valuation.
- Sales and dispatch register.
- Supplier statements.
- Customer statements.
- Outstanding ageing.
- Daily cashbook.
- Bankbook.
- Expense register.
- Vehicle maintenance report.
- Attendance report.
- Gross-margin report.
- Audit log.

- Stock-count variance report.

Requirements:
- Every KPI must be calculated from authoritative persisted records.
- All monetary figures must use consistent currency and rounding rules.
- Clearly label draft versus posted transactions.
- Distinguish invoiced sales from collected cash.
- Distinguish gross margin from net profit.
- Provide CSV/Excel exports and print-friendly reports.
- Do not use static hardcoded dashboard totals.

15. DATABASE AND DATA INTEGRITY

Create normalized PostgreSQL tables and migrations for at least:

- parties
- materials
- material_grades
- rate_history
- yards/locations
- vehicles
- vehicle_documents
- collection_jobs
- weighments
- purchase_headers
- purchase_lines
- inventory_lots
- stock_movements
- sales_headers
- sales_lines
- sales_lot_allocations
- invoices
- invoice_lines or immutable invoice snapshots
- payments
- payment_allocations
- cash_bank_accounts

- account_transactions
- expenses
- employees
- attendance_entries
- document_attachments
- approvals
- audit_logs
- users and role assignments

Refine the schema as necessary after inspecting the repository and actual workflow.

Implement:
- Primary and foreign keys.
- Appropriate unique constraints.
- Check constraints.
- Decimal numeric types for weight and money.
- Explicit transaction timestamps and effective dates.
- Database indexes for commonly filtered columns.
- Transactional posting.
- Idempotency protections for critical operations.
- Safe invoice numbering.
- Auditable opening balances.
- Reversal references.
- Row-level security or equivalent authorization where appropriate.

Avoid floating-point arithmetic for financial calculations. Use suitable decimal database types and deterministic rounding rules.

Do not store the only copy of a critical balance as a manually editable number. Derived balances must reconcile to their source ledgers.

16. SECURITY AND PERMISSIONS

Implement role-based access control.

Suggested roles:

- Owner/Admin.
- Manager.
- Purchase Operator.
- Weighment Operator.
- Sales Operator.
- Accounts Operator.
- Store Operator.
- Attendance/HR Operator.
- Read-only Auditor.

Roles and permissions must be configurable.

Enforce authorization on the server or database, not only by hiding buttons in React.

Security requirements:
- Secure authentication.
- Session and access-token handling.
- Least-privilege access.
- Server-side validation.
- Protected document storage.
- Input validation and safe error handling.
- Rate limiting where appropriate.
- Audit logging.
- No secrets in client bundles or committed configuration.
- Backups and restoration testing.
- No production financial transactions through public test endpoints.

Do not allow ordinary operators to alter audit logs, erase posted transactions or approve their own sensitive exceptions unless an explicit policy permits it.

17. AUTOMATION

Implement automation only after the underlying data is reliable.

Priority automations:
1. Purchase entry creates stock inward upon authorized posting.

2. Sales dispatch creates stock outward upon authorized posting.
3. Invoice generation fills validated transaction data automatically.
4. Payment posting updates relevant account and outstanding balances.
5. Material balances update from stock movements.
6. Low-stock alerts where minimum thresholds are configured.
7. Vehicle-document expiry reminders.
8. Outstanding-payment reminders.
9. Daily business summary.
10. Daily reconciliation exceptions report.
11. Exportable monthly reports.

Provide configurable notification thresholds and delivery channels.

Do not assume WhatsApp, SMS, email or external messaging credentials exist. Implement in-app notifications first and document any external integration prerequisites.

18. DATA MIGRATION

The current business uses paper registers and Excel.

Build a controlled migration workflow:
- Import suppliers/customers.
- Import material masters.
- Import verified opening stock.
- Import opening supplier/customer balances.
- Import cash/bank opening balances.
- Import vehicle and employee records where appropriate.

Requirements:
- CSV/XLSX upload preview.
- Column mapping.
- Data validation.
- Duplicate detection.
- Error report downloadable by the user.
- Dry-run mode.
- Explicit import confirmation.

- Import batch reference.
- Audit trail.
- Reconciliation summary.

Do not automatically trust every historical row or derive a complete historical inventory ledger from incomplete registers.

If opening quantities or balances cannot be verified, flag the discrepancy rather than silently inventing a correction.

19. TESTING REQUIREMENTS

Create automated tests for business-critical operations.

Weighment:
- Correct gross/tare/net calculation.
- Tare greater than gross rejected.
- Negative or invalid values rejected.
- Decimal precision handled correctly.
- Duplicate slip warnings.

Purchases:
- Draft does not change stock.
- Posting adds the correct quantity once.
- Duplicate posting is prevented.
- Approval and permission rules are enforced.

Inventory:
- Inward/outward balances reconcile.
- Negative stock is rejected.
- Lot allocation follows FIFO where configured.
- Transfers preserve total stock.
- Reversals restore the correct quantity.
- Concurrent dispatches cannot oversell stock.

Sales:

- Correct selling amount.
- Correct stock allocation.
- Correct cost-of-goods-sold calculation.
- Invoice numbering is unique.
- Duplicate invoice generation is prevented.

Payments:
- Partial payments work.
- Multiple allocations work.
- Over-allocation is rejected unless explicitly supported.
- Cash/bank account balances reconcile.
- Internal transfers do not create income or expense.

Invoices:
- PDF totals match persisted invoice data.
- Correct rounding.
- Correct line-item rendering.
- Long descriptions and multiple pages handled.
- Issued invoice snapshots remain unchanged after master-data edits.

Security:
- Unauthorized API requests are rejected.
- Users cannot bypass approval through direct requests.
- Restricted documents cannot be accessed without permission.

Migration:
- Invalid rows are rejected or quarantined.
- Import dry runs do not modify production records.
- Imported balances reconcile.

Run:
- Unit tests.
- Integration tests.
- End-to-end tests.
- TypeScript checks.
- Linting.

- Production build.

Do not report all tests as passing unless the commands actually ran and the results support that claim.

20. IMPLEMENTATION PHASES

PHASE 0 — REPOSITORY AUDIT AND FOUNDATION
- Inspect existing files and the supplied documents.
- Document assumptions and architectural decisions.
- Configure the project, authentication and database migrations.
- Implement shared layout, error handling and audit foundations.
- Add `PROJECT_STATUS.md`.

PHASE 1 — CORE SCRAP OPERATIONS
- Parties and materials.
- Purchases and collections.
- Weighment.
- Inventory lots and stock ledger.
- Stock reconciliation.

PHASE 2 — SALES AND FINANCE
- Sales and dispatch.
- Invoice PDF generation.
- Supplier/customer ledgers.
- Payments and allocations.
- Cashbook, bankbook and expenses.
- Financial controls.

PHASE 3 — PILOT AND MIGRATION
- Import validated opening balances.
- Test actual business scenarios.
- Compare paper/Excel registers against the application.
- Reconcile stock and financial records.
- Fix critical defects before production.

PHASE 4 — SUPPORTING OPERATIONS
- Vehicle management.
- Document expiry alerts.
- Employee attendance.
- Advanced reports.
- Scheduled notifications.

PHASE 5 — HARDENING AND RELEASE
- Security review.
- Backup and restore test.
- Performance testing.
- Production deployment documentation.
- Operator training guide.
- Release checklist and known limitations.

At the end of each phase:
1. Implement the actual functionality.
2. Run relevant tests.
3. Fix discovered failures.
4. Update `PROJECT_STATUS.md`.
5. List changed files.
6. Record test results.
7. Identify remaining blockers.
8. Continue to the next phase when safe.

Do not stop at a phase boundary merely to present a report if the next phase can be implemented safely within the available environment and scope.

21. DEFINITION OF DONE

The initial operational release is complete only when:

- An authorized user can create and post a purchase.
- Verified purchase posting creates the correct stock movements.
- Collection transactions can be recorded separately from estimates.
- Manual weighment and weighbridge-slip data can be entered.

- An authorized user can create and dispatch a sale.
- Dispatch consumes the correct inventory quantity.
- Sales invoice PDFs can be generated using the approved document design.
- Partial supplier payments and customer receipts reconcile.
- Cash and bank balances reconcile to posted account transactions.
- Stock reports match the stock ledger.
- Purchase, sales, payment and audit records are traceable.
- Critical business tests pass.
- The production build succeeds.
- Backup, migration and deployment instructions exist.
- No unresolved critical security or data-integrity defect remains.

A visually complete interface without these functions is not a completed ERP.

22. REQUIRED FINAL DELIVERABLES

Create or update:
- Working application source code.
- Database schema and versioned migrations.
- Environment variable example file.
- Seed data for development only.
- API and business-rule documentation.
- Automated tests.
- Invoice PDF renderer.
- Data import/export tools.
- `README.md`.
- `PROJECT_STATUS.md`.
- `ARCHITECTURE.md`.
- `DATABASE_SCHEMA.md`.
- `BUSINESS_RULES.md`.
- `TEST_REPORT.md`.
- `DEPLOYMENT.md`.
- `USER_GUIDE.md`.

At completion, report:
1. What is implemented and verified.

2. What is partially implemented.
3. What remains unimplemented.
4. Exact test commands and results.
5. Production build result.
6. Database setup and migration instructions.
7. Required external credentials or integrations.
8. Known risks and limitations.
9. Steps required before real company data is used.

FINAL EXECUTION INSTRUCTION

Start by inspecting the current repository and the supplied invoice and letterhead files. Then implement the application in the current workspace, beginning with Phase 0.

Do not replace implementation with a long plan. Do not invent business policies that have not been confirmed. Record unresolved decisions and use clearly documented, configurable defaults where safe.

Prioritize data integrity, correct stock movements, accurate financial calculations, invoice fidelity and reduced manual workload over decorative UI features.

Build a real, testable Yasoth Enterprises Scrap ERP that can be safely piloted against the company's existing paper and Excel records.

How to run it in Google Antigravity

1. Create or open a dedicated project folder for Yasoth Enterprises.
2. Place `TEJAS_INVOICE_TEMPLATE.xlsm.xlsx` and `yasoth Enterprises .pdf` inside the project workspace so the agent can inspect them.
3. Open the workspace in Google Antigravity and enable the agent's terminal and file-editing capabilities.
4. Paste the entire master prompt above into the agent.
5. Let it inspect the repository and implement the first phase. Review database migrations, environment configuration and generated files before authorizing any production deployment.

Important: Don't let the agent connect the initial build directly to live financial records. Use a development database first, test purchase-to-stock-to-sale reconciliation, and migrate verified opening balances only after the workflow passes.